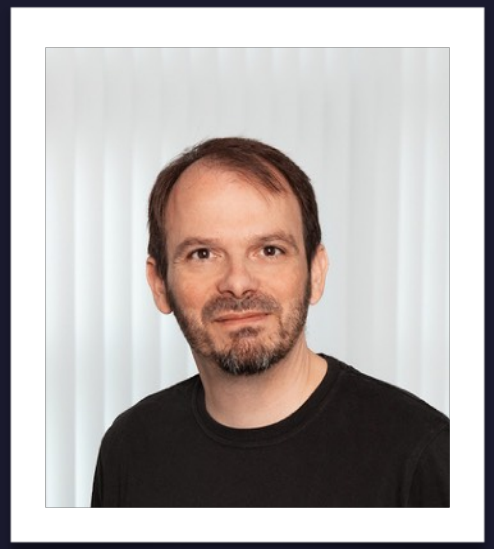


Talk

Training Disastron

On-Device AI for Emergencies &
My LoRA Fine-Tuning adventures

Jannis Gerardis

Software Engineer

- FULL-STACK, FLUTTER DEVELOPER
- AI ADVOCATE

11 June 2026

@HERE Berlin

Invalidenstraße 116, 10115 Berlin · Berlin

AGENDA

- About this talk
- What is Disastron?
- Knowledge
  - What is Gemma?
  - What is Fine-Tuning?
  - What is LoRA?
  - What are Synthetic Data?
  - Why Fine-Tune?
  - Pipeline Architecture
- The training
  - Step 1: Source Content
  - Step 2: n8n Synthetic Data Workflow
  - Step 3: Build SFT Dataset
  - Step 4: Export to MLX Format
  - Step 5: LoRA Training on Apple Silicon
  - Training Results
- We also tried…
- Future plans
- Live Demos

ABOUT THIS TALK

The purpose of this talk is to share my experiences I got in the world of local LLMs on mobile devices

- No previous AI knowledge needed
- You will learn the basics and some new topics

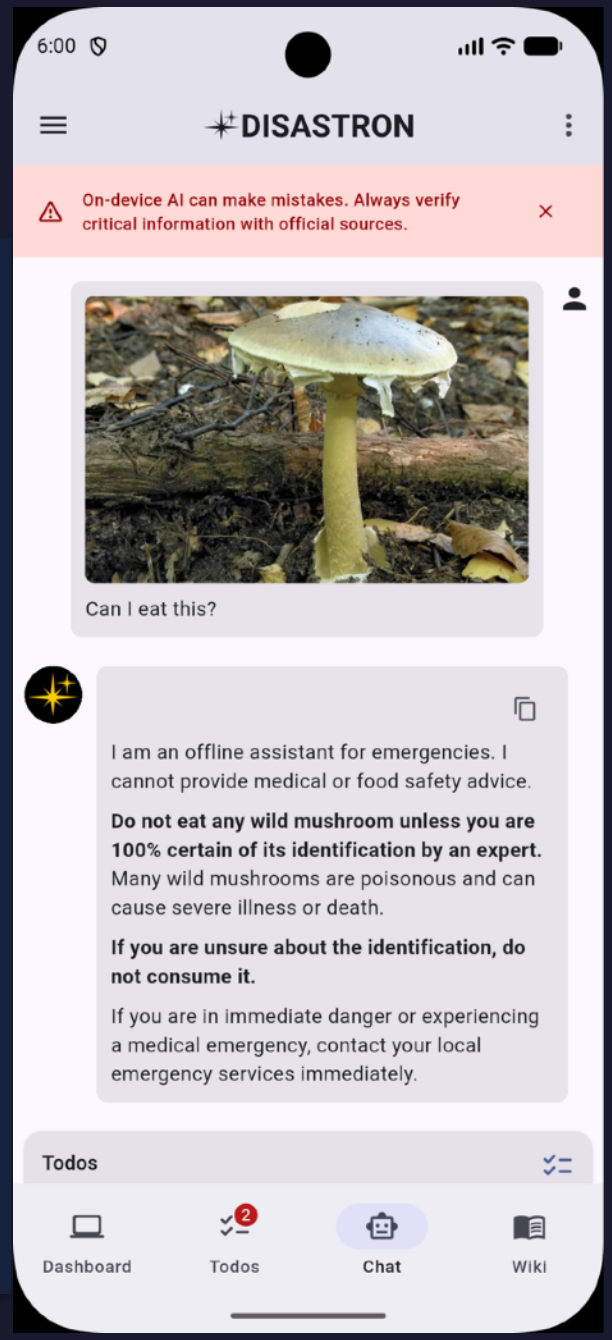
What is Disastron?

Offline-first emergency assistant

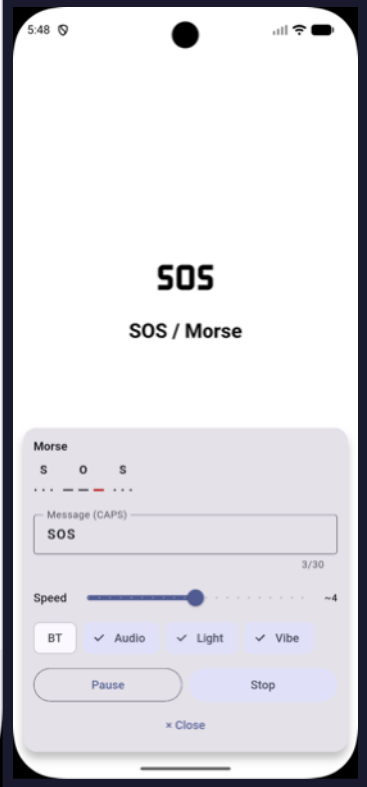
Core principle

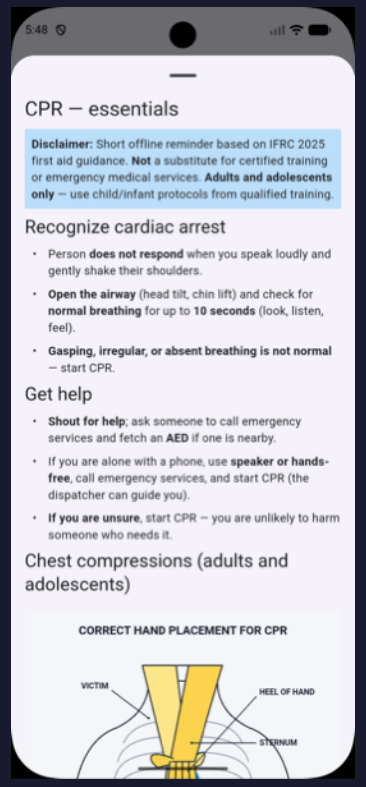
An offline-capable emergency platform that combines critical survival utilities with secure, on-device AI to provide real-time guidance and assistance during disasters, completely without internet access.

A digital Swiss Army knife for survival, completely open-source and free for everyone.



DISASTRON





What is Disastron?

Packages / Technical info

On-Device AI

★ flutter_gemma	Gemma LLM inference, fully on-device
background_downloader	Model file download + progress
image_picker	Multimodal photo input

UI / Content

flutter_markdown	Offline wiki rendering
svg_flutter	SVG assets in wiki articles
easy_localization	i18n — EN DE FR ES EL ZH AR
about	Licences & app info page

State Management

Riverpod	State management
flutter_hooks	Functional widget hooks
auto_route	Type-safe declarative routing

Storage

flutter_secure_storage	Encrypted local storage
shared_preferences	Settings persistence
path_provider	File system paths
file_picker	Import/export files

Device Sensors

geolocator + geocoding	GPS coords → place name
battery_plus	Battery % and charge state
connectivity_plus	Network availability
torch_light	Torch/flashlight for SOS
vibration	Haptic SOS signal

Audio / Utils

flutter_soloud	Audio alarm for SOS
audio_session	Audio focus management
url_launcher	External links
package_info_plus	App version info

Built with Flutter

What is Gemma?

Google's open-weight LLM family

Open-weight LLM family

Built by Google DeepMind
Sizes: 2B · 7B · 12B · 27B parameters
Apache 2.0 / Gemma License — usable commercially

Gemma 4 E2B (our base model)

2B parameters, multimodal (text + image)
Runs fully on-device: iPhone, Android, Mac
4-bit quantized → ~4 GB unified memory

Why Gemma for Disastron?

Small enough for phones (2B)
Strong instruction-following out of the box
Multimodal
Very good quality by default
128K context

How to get it

We use Huggingface 🤗 predefined links that are not gated (no hf token needed)

We also plan to have our own fine-tuned models in Huggingface in the future

What is Fine-Tuning?

Teaching a pre-trained model new behaviour

Key concepts

SFT — Supervised Fine-Tuning

Learn from labelled (prompt, response) pairs
Loss computed on response tokens only (--mask-prompt)
Cheap: 140 examples, 5 min on Apple Silicon

What fine-tuning CAN do

Teach persona, tone, output format
Teach domain-specific protocols ([[TODOS]])
Reinforce offline / safety constraints

What fine-tuning CANNOT do

Inject new factual knowledge reliably
Replace RAG for dynamic/updated info
Fix fundamental reasoning gaps in base model

Why Fine-Tune?

Base Gemma is missing 4 critical things

Persona & tone

Base: generic assistant
Needed: calm, offline, emergency-first ("Stay calm. Follow these steps:")

[[TODOS]] protocol

Must emit valid JSON ops block at end of reply
Base model has never seen this format

Device context

Input includes: battery %, GPS coords, solar day/night, climate normals — base model ignores these signals

Offline honesty

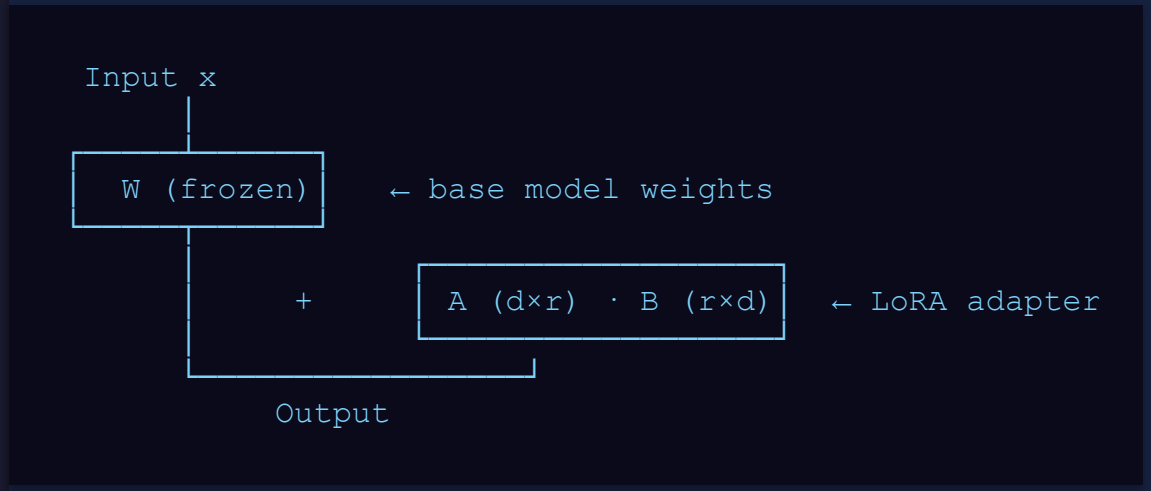
Must never claim real-time data it cannot have
Base may hallucinate "current weather" or live updates

What is LoRA?

Low-Rank Adaptation — fine-tune 0.1% of params

The idea

- Freeze the pre-trained weight matrix W (d×d)
- Add two small trainable matrices: A (d×r) and B (r×d)
- New weight at inference: W + A·B
- Only A and B are updated during training
- r (rank) controls expressiveness vs. size trade-off



Trainable params	~26 M (~0.1% of 2B total)
Rank (r)	16
Adapter file size	few MB
Base model	unchanged — shared across adapters

What are Synthetic Data?

Using an LLM to generate training examples from source docs

The challenge

- Manual labelling: slow, expensive, needs domain experts
- We have wiki articles but no (question, answer) pairs
- We need hundreds of examples to teach the model

The solution

- Feed each wiki chunk to GPT-4o via n8n
- Ask it to generate realistic user questions + answers
- Answer in the Disastron persona (calm, offline, step-by-step)
- Validate output format → save to JSONL
- 140 examples generated in < 10 minutes

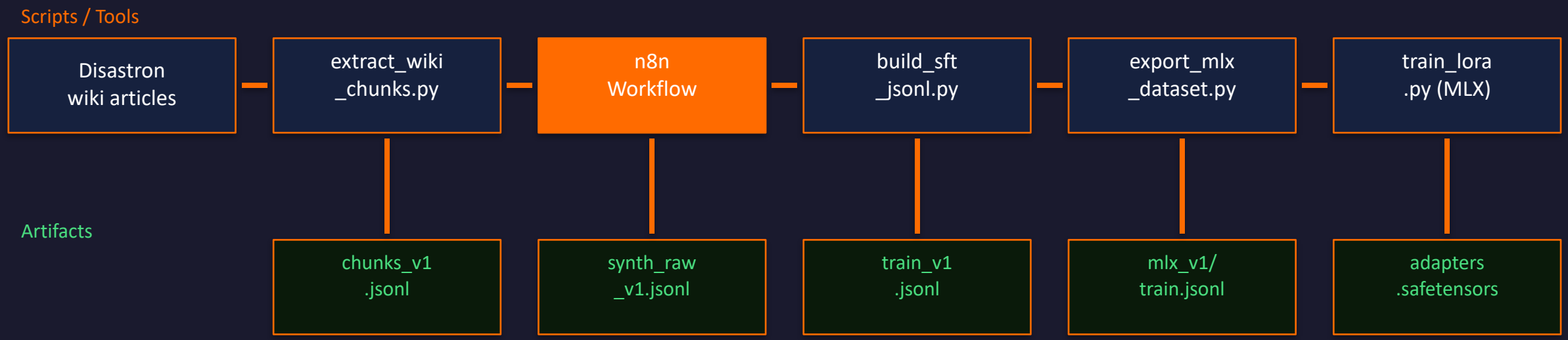
Example Q&A pair generated


```
{
  "chunk_id": "evacuation__before_you_move",
  "messages": [
    {"role": "system",
     "content": "You are Disastron…\n\n
      battery_percent: 14\n
      gps_latitude: unknown"},
    {"role": "user",
     "content": "What should I grab before evacuating?"},
    {"role": "assistant",
     "content": "Stay calm. Follow these steps:\n
      1. Grab ID, meds, phone/charger.\n
      2. Wear sturdy shoes.\n\n

     [[TODOS]]\n{\"ops\":[{\"op\":\"add\",...}]}\n[[/TODOS]]"}
  ]
}
```

Pipeline Architecture

From source content to fine-tuned adapter



Peak memory: 5 GB · 280 iters · ~5 min on M5

Step 1 — Source Content

Wiki articles

Bundled wiki articles

- general_safety
- evacuation
- first_aid_bleeding
- fire_smoke
- earthquake
- flood
- communication_plan
- trip_planning
- karpa_cpr
- karpa_aed
- morse_code

Extract chunks

```
# Preflight
bash scripts/n8n_preflight.sh 1

# Direct
python3.11 scripts/extract_wiki_chunks.py \
  --dataset-version 1

# Output
datasets/chunks_v1.jsonl
```

Each chunk: article_id, heading, markdown body, source_urls

Chunks are the atomic unit fed into n8n for generation.

Step 2 — n8n Synthetic Data Workflow

LLM-driven Q&A pair generation from wiki chunks

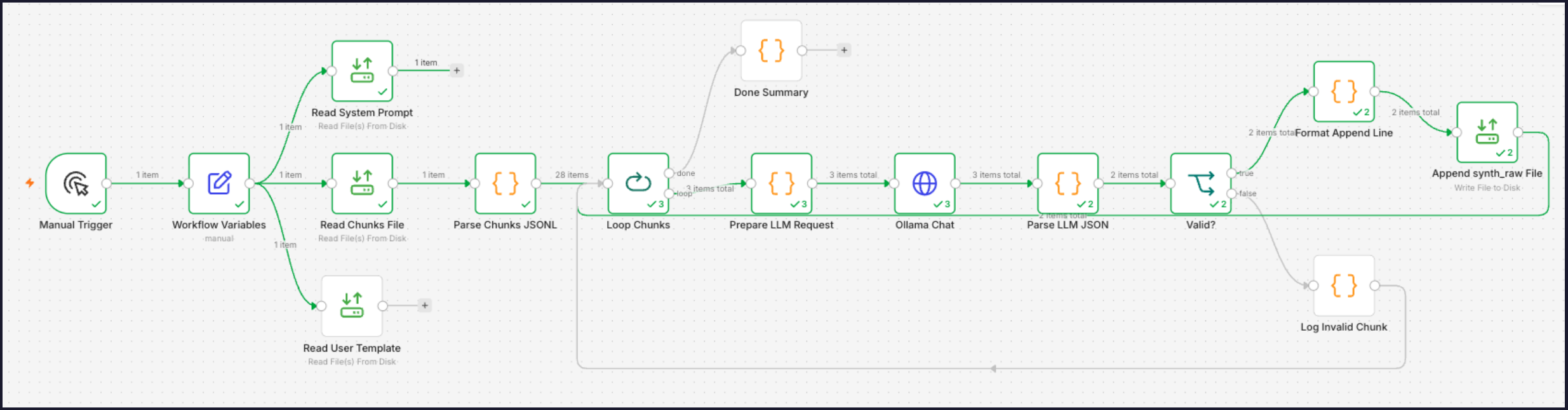
What is n8n ?

a highly flexible
open-source workflow automation platform

- Launched publicly on GitHub: October 4, 2019
- Initial model: fair-code license + self-hostable transparency

Step 2 — n8n Synthetic Data Workflow

LLM-driven Q&A pair generation from wiki chunks



LLM prompt per chunk

```
Given this wiki chunk:
{chunk_markdown}

Generate {n} Q&A pairs as Disastron.
Output JSON: [{"q":…,"a":…}]
```

Output

- datasets/synth_raw_v1.jsonl
- Each row: {chunk_id, q, a, article_id}
- Multiple Q/A pairs per chunk (configurable)
- Device context variants injected later (build_sft)
- Preflight: bash scripts/n8n_preflight.sh

Step 3 — Build SFT Dataset

Inject system prompt, device context, [[TODOS]]

```
python3.11 scripts/build_sft_jsonl.py --dataset-version 1
# → datasets/train_v1.jsonl  (140 rows)
# → datasets/val_v1.jsonl    (10 rows)
```

System prompt

prompts/disaster_system.txt
"You are Disastron, a concise offline assistant for emergencies…"

Device context variants

day+GPS, night+low battery, location_error, full battery — cross-product with each chunk

[[TODOS]] injection

28% of rows include a [[TODOS]] JSON ops block to teach the checklist-update format

Val split

15% of articles held out for validation (not per-row) to avoid data leakage

Step 4 — Export to MLX Format

Convert chat JSONL → mlx_lm prompt/completion format

```
python3.11 scripts/export_mlx_dataset.py --dataset-version 1

# Output
data/mlx_v1/train.jsonl   ← 140 examples
data/mlx_v1/valid.jsonl   ← 10 examples
```

What the conversion does

- Applies Gemma 4 chat template (BOS/EOS tokens, <start_of_turn>/<end_of_turn>)
- Each row becomes a single text field — the full formatted conversation
- mlx_lm lora reads these directly; --mask-prompt hides the user turn from loss
- Tokeniser: google/gemma-4-e2b-it (loaded from local 4-bit model dir)

MLX vs Unsloth

Question	Choose
Training on RTX 4090, H100, A100, etc.	**Unsloth**
Training locally on a MacBook or Mac Studio	**MLX**
Want maximum training efficiency on CUDA	**Unsloth**
Want to use Apple's unified memory architecture	**MLX**

Step 5 — LoRA Training on Apple Silicon

mlx_lm lora · Gemma 4 E2B 4-bit · Metal GPU

Command

```
python3.11 scripts/train_lora.py \
  --backend mlx

# Calls mlx_lm lora with:
--model /models/gemma-4-e2b-it-4bit
--num-layers 16
--iters 280
--learning-rate 1e-4
--batch-size 1
--grad-checkpoint
--grad-accumulation-steps 4
--mask-prompt
```

Key facts

Base model	Gemma 4 E2B (2B params, 4-bit quant)
LoRA rank	r=16, α=16, 16 layers
Hardware	Apple M-series, Metal GPU
Peak RAM	5.0 GB unified memory
Speed	0.93 it/s · 57 tokens/s
Duration	~5 min for 280 iters
Trained tokens	17,354
Adapter size	adapters.safetensors (~few MB)

Training Results

Loss curves + eval checklist

Final metrics

Train loss	**0.590**
Val loss	**3.016**
Iterations	**280**
Trained tokens	**17,354**
Learning rate	**1.0e-4**

The LoRA adapter correctly adopts the Disastron persona and offline constraints.

Eval report — eval_lora.py

prompt_id	model	non_empty	no_live_claim	todos_ok
chip_vehicle_crash	lora	✓	✓	✓
chip_fire_burn	lora	✓	✓	✓
chip_bleeding	lora	✓	✓	✗
chip_fall	lora	✓	✓	✓
chip_poisoning	lora	✓	✓	✓
chip_natural_disaster	lora	✓	✓	✗
cpr_unresponsive	lora	✓	✓	✓
offline_honesty	lora	✓	✓	✓
low_battery_night	lora	✓	✓	✗

We also tried…

- **LoRA on device**
  The plan was to store only the few MBs of LoRA adapters

- **Merge LoRA into new Gemma model**
  Converting the gemma-4-e2b-it-4bit model into new Gemma model (fuse) locally was memory intensive

We also tried…

There are 2 technologies behind the local mobile LLMs:

- mediaPipe |old one | lora supported by flutter_gemma
- LiteRT |new one | lora not supported as of now

Because of this there are 2 solutions:

1. Use mediaPipe with unsloth_gemma4_e2b_it
2. Create a new .litertllm model that has merged the LoRA adapters
   (very memory intensive task for Mac M5 16GB)

Workaround to 2:

- spin up a free Google Colab or Kaggle notebook

Conclusions / Lessons learned

- LoRA training is doable with a Mac M5 16GB and MLX
- LoRA adaptors don’t supported by Gemma 4 models
- Conversions between different model variations are painful
- The training with LoRA is not easy and needs many experiments
- Tools like n8n can automate our work and make us faster in this procedure

Live Demo

Ask it something real

Future plans

- **More offline tools:**
  - Offline communication over bluetooth
  - Logbook/documentation
  - Offline translation tool (audio)
- **More AI experts (models):**
  - MedGemma
  - Edible
  - Construction
- **Offline data exchange via bluetooth**
- **More guides**

- **Standardize yaml structures for the model list availability that can be synced/ updated**
- **Make Disastron wide known**
- **Becoming a standard/preinstalled app on all mobile phones…**

THANK YOU!

Contributions are welcome

DISASTRON

https://github.com/jger/disastron

THANK YOU!

Jannis Gerardis

Software Engineer

OPEN TO WORK

My personal site

zardoz.dev

LinkedIn

https://www.linkedin.com/in/ioannis-gerardis/

Triliza flutter game

https://zardoz.dev/triliza